Отношения между классами и объектами

Александра Волосова,
к.т.н., доцент кафедры
ИУ5

Презентация разработана в рамках гранта «Грант на обучение студентов по образовательным программам высшего образования для топ-специалистов в сфере информационных технологий. Договор № 70-2025-000850 с АНО «Аналитический центр при Правительстве Российской Федерации»

Часть 1

Отношение композиции

Отношения между классами

- ассоциация (**is-a отношение**)
- агрегация (**has-a отношение**)
- композиция (**part-of отношение**)
- наследование

Для двух объектов Foo и Bar можно определить взаимосвязи между ними

- **Ассоциация:** у Foo — семантическая связь с объектом Bar

```
public class Foo {
private Bar bar; };
```

- **Композиция** — Foo являюсь владельцем предмета и несет ответственность за его жизнь. Когда уничтожается Foo, уничтожается и Bar

```
public class Foo {
private Bar bar = new Bar();
}
```

- **Агрегация** — Foo заимствует Bar. Когда Foo уничтожается, Bar продолжать жить

```
public class Foo {
private Bar bar; Foo(Bar bar) {
  this.bar = bar; }
}
```

Ассоциация

- один объект **связан** с другим, но они не обязательно являются частями друг друга, и их жизненные циклы никак не связаны

Пример:

```
#include <iostream>
#include <string>
class Doctor {
private:
   std::string name;
public:
   Doctor(std::string n) : name(n) {}
   void treatPatient() {
      std::cout << "Доктор " << name << " лечит.\n";
   } };
class Patient {
private:
   std::string name;
//Ассоциация (связь, но не владение)
   Doctor* myDoctor;
public:
   Patient(std::string n, Doctor* doc) : name(n), myDoctor(doc) {}
      void askForHelp() {
      std::cout << "Пациент " << name << " просит помощи у ";
      myDoctor->treatPatient(); // Используем связанного доктора
   } };
```

```
int main() {
   Doctor drHouse("Хаус");
   // Пациент связан с доктором
   Patient patient1("Пациент 1", &drHouse);
   // Другой пациент связан с тем же доктором
   Patient patient2("Пациент 2", &drHouse);
   patient1.askForHelp();
   patient2.askForHelp();
   // Когда объекты  patient1 и patient2 удалятся,
   // объект  drHouse продолжит жить.
   return 0; }
```

```
Пациент Пациент 1 просит помощи у Доктор Хаус лечит.
Пациент Пациент 2 просит помощи у Доктор Хаус лечит.
```

Существует **два** основных **подтипа** композиции объектов:

1. Композиция: Отношения композиции — это отношения части и целого, где часть может быть частью только одного объекта одновременно. Это означает, что часть создается при создании объекта и уничтожается при его уничтожении. Чтобы считаться композицией, для объекта и его части должны выполняться следующие утверждения:

- Часть (член) является частью объекта (класса).
- Данная часть (член) может принадлежать только одному объекту (классу).
- Существование данной части (члена) управляется объектом (классом).
- Данная часть (член) не знает о существовании объекта (класса).

Существуют некоторые вариации правила создания и уничтожения частей:

- В композиции создание некоторых частей может быть отложено до тех пор, пока они не понадобятся.
- Композиция может использовать предоставленную ей в качестве исходного материала часть, а не создавать её самостоятельно.
- Композиция может делегировать уничтожение своих частей какому-либо другому объекту.

Композиция

- построения **сложных объектов** из **более простых** называется **композицией объектов**

```
class Point {
private:
    int x;
    int y;
};
class Location {
    Private : Point Source;
    Point Destination
};
```

Композиция

```
#include <iostream>
class One {
private:
   int num;
public:
   void set(int i)
   {  num = i;  }
   int get()
   {  return num; }
};
class Two {
public:
   One O;
   void show()
   {std:: cout << "\n Number = "
            << O.get();
   } };
```

```
int main()
{
   Two T;
 //Выполнить некоторую операция, используя
// объект One в этом классе
   T.O.set(100);
   T.show();
return 0;
}
```

```
Number = 100
```

Для доступа к члену класса One необходимо использовать объект класса Two: **T.O.set(100)**

num является закрытым членом класса One, для доступа к его значению необходимо использовать открытую функцию

Композиция

композиция объектов используется для объектов, которые имеют **отношение «имеет (has-a)»** друг с другом. Поэтому сложный объект называется родительским объектом, тогда как более простой объект часто называют дочерним объектом.

Синтаксис:

```
class A
{ };
class B
{  A objA;
   public:
    B(arg-list) : objA(arg-list1);
};
```

Класс B использует объекты класса A в качестве своих членов данных. Следовательно, B — это сложный класс, использующий простой класс A

```
#include <iostream>
using namespace std;
class A {
public:
   int x;
   A() { x = 0; }
   A(int a)
   {
      cout << "Constructor A(int a) is invoked"
<< endl;
      x = a;
   }
};
```

```
Constructor A(int a) is invoked
Data in object of class B = «25
Data in member object of class A in class B = 25
```

```
class B {
   int data;
   A objA;
public:
    B(int a)
       : objA(a)
   {   data = a; }
void display()
   {   cout << "Data in object of class B = «"
<< data << endl;
       cout << "Data in member object of "
           << "class A in class B = " << objA.x;
   }
};

int main()
{B objb(25);
    objb.display();
   return 0; }
```

Преимущества объектной композиции:

Использование композиции объектов может обеспечить следующие преимущества:

Повторное использование существующего кода: Композиция объектов позволяет повторно использовать существующий код без необходимости моделирования **отношения «is-a»**, как это обычно делается в наследовании.

Чистый дизайн API: Использование композиции объектов помогает создавать чистые и компактные API. Это связано с тем, что при композиции классов проще решить, станет ли ссылочный класс частью API или будет скрыт.

Изменение реализации класса, используемого в композиции, без привлечения внешних клиентов: композиция также позволяет упростить изменение и адаптацию кода при необходимости. Внутренние классы можно изменять без каких-либо побочных эффектов, и изменения могут обрабатываться внутри системы.

Часть 2

Отношение агрегации

Делегирование

2. Агрегация: Агрегация также представляет собой **отношение «часть-целое»**, но в данном случае **части** могут принадлежать **более чем одному объекту** одновременно, и целый объект не несет ответственности за существование частей. Чтобы считаться агрегацией, целый объект и его части должны иметь следующие отношения:

- Эта часть (член) является частью объекта (класса)
- Данная часть (член) может одновременно принадлежать более чем одному объекту (классу)
- Существование данной части (элемента) не управляется объектом (классом)
- Данная часть (член) не знает о существовании объекта (класса)

Агрегация

```
class PartClass{
//instance variables
//instance methods
}
class Whole{
PartClass* partclass;
}
```

Пример: композиция и агрегация

```
// Композиция: Engine создается и уничтожается с Car
class Car {
private:
    Engine engine;      // Композиция
    std::string model;
public:
    Car() : engine(200) {}  // Engine создается здесь
    // Деструктор Car автоматически уничтожает engine
};

// Агрегация: Wheel может существовать без Car
class Wheel { /* ... */ };
class Car2 {
private:
    Wheel* wheels[4];   // Агрегация
    Engine& engineRef;  // Агрегация (ссылка)
public:
    Car2(Wheel* w[], Engine& e) : engineRef(e) {
        for (int i = 0; i < 4; ++i) wheels[i] = w[i];
    }
    // Деструктор не уничтожает Wheel объекты};
```

Композиция и агрегация

- Композиция: часть-целое (part-of) с общим временем жизни
- Агрегация: часть-целое (part-of) с независимым временем жизни
- Композиция: объект создается и уничтожается с контейнером
- Агрегация: объект может существовать независимо

При помощи агрегации и композиции можно реализовать делегирование

- **Агрегация и Композиция - реализация отношения «часть-целое»**
- **Делегирование - принцип**, при котором объект **передает (делегирует)** выполнение какой-либо задачи другому объекту (своей "части")

Делегирование объектов

- означает использование **объекта одного класса** в качестве **члена другого класса**. Это называется делегированием объектов. Делегирование может быть альтернативой наследованию, но при наследовании существует отношение «является (is-a)», а при делегировании отношения наследования между классами отсутствуют

```
#include <iostream>
using namespace std;
class First {
public:
    void print() { cout << "The Delegate"; }
};
class Second {
First ob;
public:
    void print() { ob.print(); }
};
```

```
int main()
{
    Second ob1;
    ob1.print();
    return 0;
}
```

Пример: **реализация делегирования через композицию**

```
class PrintHead {
public:
    void printText(const std::string& text) {
        std::cout << "Печатаю: " << text << std::endl;
    } };
class Printer {
private:
    PrintHead head;
// Композиция: PrintHead — часть принтера
public:
    void print(const std::string& doc) {
        // Делегирование: функции печать части принтера
        head.printText(doc);
    }
};
```

```
int main() {
    Printer hp;
    hp.print("Hello");
// Принтер делегировал печать
    return 0; }
```

Пример: **реализация делегирования через агрегацию**

```
class Worker {
public:
   void doWork(const std::string& task) {
      std::cout << "Работник выполняет: "
                << task << std::endl;
   } };
class Manager {
private:
// Агрегация: работник пришел извне
   Worker* subordinate;
public:
   Manager(Worker* w) : subordinate(w) {}
    void delegateWork(const std::string& task)
{ // Делегирование: менеджер просит работника
      if (subordinate) {
         subordinate->doWork(task);
         } } };
```

```
int main() {
    Worker john;
// Агрегация (передать ссылку)
    Manager boss(&john);
// Делегирование
   boss.delegateWork("Отчет");
return 0;
}
```

```
Работник выполняет: Отчет
```

Преимущества делегирования объектов:

- Класс повторного использования можно использовать по разному, не изменяя его объявления
- Несколько классов, использующих одни и те же переменные, могут делать это одновременно
- Переменные и методы повторного использования, а также повторно используемые классы могут находиться на разных компьютерах
- Делегирование работает является альтернативой наследования. Что используется в языках с одиночным наследованием (Java, C#)

Делегирование:

- **Гибкость:** Легко комбинировать поведение, не ломая старый код
- **Разделение:** Легко организовать совместный доступ к ресурсам
- **Распределенность:** Легко абстрагироваться от физического расположения объекта
- **Свобода:** Не зависит от ограничений иерархии наследования

Композиция объектов	Делегирование объектов
Взаимоотношения между объектами	Передача функций одного объекта другому
В композиции методы, используемые для создания внутреннего объекта, могут применяться только в личных целях и не должны выставляться на всеобщее обозрение	Делегирование предполагает реэкспорт методов
Композиция влияет на жизненный цикл объекта: родительский объект владеет дочерним, а дочерний объект не может существовать самостоятельно	Делегирование не влияет на жизненный цикл

Часть 3

Принцип SOLID

Обертывание

Принципы SOLID

- аббревиатура пяти принципов объектно-ориентированного проектирования, предложенная Робертом Мартином в начале 2000-х:

- **S** - Single Responsibility Principle (Принцип единственной ответственности)

  Класс должен делать что-то одно

- **O** - Open/Closed Principle (Принцип открытости/закрытости)

  Классы открыты для расширения, но закрыты для изменения

- **L** - Liskov Substitution Principle (Принцип подстановки Лисков)

  Объекты можно заменить наследниками без нарушения работы программы

- **I** - Interface Segregation Principle (Принцип разделения интерфейсов)

  Много маленьких интерфейсов лучше одного большого

- **D** - Dependency Inversion Principle (Принцип инверсии зависимостей)

  Зависимость от абстракций, а не от конкретных классов

S - Принцип единственной ответственности

Неудачно:

```
class Bad {
    string name;
    double salary;
public:
    // Работа с данными
    void setName(string n) {
name = n; }
    // Расчет зарплаты
    double calculateTax() {
        return salary * 0.13;
    }
// Работа с БД
    void saveToDatabase() {
        // SQL запросы
    }
    // Отправка email
    void sendEmail() {
        // SMTP протокол
    }
};
```

Хорошо:

```
class Employee {          //
Только данные
    string name;
    double salary;
};
class TaxCalculator {     //
Только расчеты
    double
calculate(Employee& e);
};
class EmployeeRepository {
// Только БД
    void save(Employee& e);
};
// Только отправка
class EmailService {void
send(string to, string msg);
};
```

Хорошо:

```
template<typename T, typename
Calc, typename Repo, typename
Email>
class EmployeeSystem {
    T employee;
    Calc calculator;
    Repo repository;
    Email emailService;
public:
    void processEmployee() {
calculator.calculate(employee);
repository.save(employee);
emailService.send(employee.get
Email(), "Done");
    }
};
```

O - Принцип открытости/закрытости

Неудачно:

```
class Bad {
public:
   double
calculateDiscount(string type,
double price) {
      if (type == "regular") {
         return price * 0.9;
      }
      else if (type == "vip") {
         return price * 0.8;
      }
 // При добавлении нового типа –
//  меняем класс!
      else if (type == "super") {
// Новый код внутри класса
         return price * 0.7;
      }
   } };
```

Хорошо:

```
class DiscountStrategy {
public:
   virtual double
calculate(double price) = 0; };
class RegularDiscount : public
DiscountStrategy {
   double calculate(double
price) override { return price *
0.9; } };
class VIPDiscount : public
DiscountStrategy {
   double calculate(double
price) override { return price *
0.8; }
};
// Новый тип добавляется снаружи!
class SuperDiscount : public
DiscountStrategy {
   double calculate(double
price) override { return price *
0.7; }
};
```

Хорошо:

```
template<typename
DiscountPolicy>
class PriceCalculator {
   DiscountPolicy discount;
 public:
   double calculate(double price) {
      return discount.apply(price);
   }
};
// Используем любой тип, имеющий метод
//apply()
struct RegularPolicy {
   double apply(double p) { return p
* 0.9; }
};
struct VIPPolicy {
   double apply(double p)
{ return p * 0.8; }
};
```

L - Принцип LSP

Обычное наследование:

```
#include <iostream>
#include <stdexcept>
class Птица {
public:
    virtual void летать() {
        std::cout << "Лечу!" << std::endl; }
     virtual ~Птица() = default; };
class Пингвин : public Птица {
public:
    // Пингвин не умеет летать, но наследует метод!
    void летать() override {
        throw std::runtime_error("Я не умею летать!"); } };
// Проблема:
void запуститьПтицуВПолёт(Птица& птица) {
    птица.летать(); // Упадет с ошибкой для пингвина! }
```

```
int main() {
    try {
        Птица птица;
    // Птица летит
        запуститьПтицуВПолёт(птица);
        Пингвин пингвин;
    // Ошибка: исключение!
        запуститьПтицуВПолёт(пингвин); }
        catch (const std::exception& e) {
        std::cout << "Ошибка: " << e.what()
                    << std::endl;
    }
    return 0;
}
```

L - Принцип LSP

```
#include <iostream>
#include <memory>
#include <vector>
// Абстрактный базовый класс
class Птица {
public:
// Чисто виртуальный метод
   virtual void двигаться() = 0;
    virtual ~Птица() = default; };
class ЛетающаяПтица : public Птица {
protected:
   virtual void летать() {
      std::cout << "Лечу по воздуху!"
                << std::endl; }
 public:
   void двигаться() override {
      летать();
   } };
class НелетающаяПтица : public Птица {
protected:
   virtual void ходить() {
std::cout << "Хожу по земле!" <<
std::endl; }
  public:
   void двигаться() override {
      ходить();
   } };
```

```
// Добавим еще примеры птиц
class Страус : public НелетающаяПтица
{
   // Наследует движение через ходьбу
};
class Орёл : public ЛетающаяПтица
{
   // Наследует движение через полет
};
class Пингвин : public НелетающаяПтица {
protected:
   void ходить() override {
      std::cout << "Хожу вразвалочку!"
                << std::endl;
   }
public:
   void плавать() {
      std::cout << "Плаваю в воде!" << std::endl;
   }
};
// Теперь все работает:
void переместитьПтицу(Птица& птица) {
   птица.двигаться(); // Работает для любой
птицы!
}
```

```
int main() {
   std::cout << "=== Правильная иерархия (LSP)
===\n" << std::endl;
    // Создаем разных птиц
   ЛетающаяПтица синица;
   НелетающаяПтица курица;
   Страус страус;
   Орёл орёл;
   Пингвин пингвин;
    // Всех можно перемещать одинаково!
   std::cout << "Синица: ";
   переместитьПтицу(синица);
   std::cout << "Курица: ";
   переместитьПтицу(курица);
   std::cout << "Страус: ";
   переместитьПтицу(страус);
    std::cout << "Орёл: ";
   переместитьПтицу(орёл);
   std::cout << "Пингвин: ";
   переместитьПтицу(пингвин);
   // Пингвин может и плавать дополнительно
   std::cout << "\nПингвин плавает: ";
   пингвин.плавать();
   return 0;
}
```

```
Синица: Лечу по воздуху!
Курица: Хожу по земле!
Страус: Хожу по земле!
Орёл: Лечу по воздуху!
Пингвин: Хожу вразвалочку!

Пингвин плавает: Плаваю в воде!
```

L - Принцип LSP – чисто виртуальные функции

```
#include <iostream>
#include <vector>
#include <memory>
class Птица {
public:
   virtual void двигаться() = 0;
   virtual ~Птица() = default;
};
class ЛетающаяПтица : public Птица {
public:
   // Чисто виртуальный метод для
летающих птиц
   virtual void летать() = 0;
    void двигаться() override {
      летать();
   }
};
class НелетающаяПтица : public Птица {
public:
   // Чисто виртуальный метод для
нелетающих птиц
   virtual void ходить() = 0;

   void двигаться() override {
      ходить();
   }
};
```

```
class Синица : public ЛетающаяПтица {
public:
   void летать() override {
      std::cout << "Синица: порхаю с ветки на ветку!" <<
std::endl;
   }
};
class Орёл : public ЛетающаяПтица {
public:
   void летать() override {
      std::cout << "Орёл: парю высоко в небе!" <<
std::endl;
   }
};
class Курица : public НелетающаяПтица {
public:
   void ходить() override {
      std::cout << "Курица: клюю зернышки во дворе!" <<
std::endl;
   }
};
class Пингвин : public НелетающаяПтица {
public:
   void ходить() override {
      std::cout << "Пингвин: переваливаюсь с боку на
бок!" << std::endl;
   }
    void плавать() {
      std::cout << "Пингвин: ныряю в океан за рыбой!" <<
std::endl;
   } };
```

```
int main() {
   std::cout << "=== Зоопарк с правильным LSP
===\n" << std::endl;
    std::vector<std::unique_ptr<Птица>> птицы;
   птицы.push_back(std::make_unique<Синица>());
   птицы.push_back(std::make_unique<Орёл>());
   птицы.push_back(std::make_unique<Курица>());
  птицы.push_back(std::make_unique<Пингвин>());
  // Все птицы двигаются через общий интерфейс
   for (const auto& птица : птицы) {
      птица->двигаться();
   }
      // Специфическое поведение пингвина
   std::cout << "\n";
   Пингвин пингвин;
   пингвин.плавать();
   return 0;
}
```

```
=== Зоопарк с правильным LSP ===

Синица: порхаю с ветки на ветку!
Орёл: парю высоко в небе!
Курица: клюю зернышки во дворе!
Пингвин: переваливаюсь с боку на бок!

Пингвин: ныряю в океан за рыбой!
```

I – Разделение интерфейсов

Неудачно:

```
class Worker {  // Огромный интерфейс
public:
   virtual void work() = 0;
   virtual void eat() = 0;
   virtual void sleep() = 0;
   virtual void code() = 0;
   virtual void design() = 0;
   virtual void manage() = 0;
};
class Robot : public Worker {  // Роботу не нужны eat/sleep!
   void work() override {}
   void eat() override { throw "Robots don't eat"; }  // ПРОБЛЕМА!
   void sleep() override { throw "Robots don't sleep"; }  // ПРОБЛЕМА!
   // ...
};
```

Хорошо:

```
class Workable {
public:
   virtual void work() = 0;
};
class Eatable {
public:
   virtual void eat() = 0;
};
class Sleepable {
public:
   virtual void sleep() = 0;
};
class Human : public Workable, public Eatable, public Sleepable {
   void work() override {}
   void eat() override {}
   void sleep() override {}
};
class Robot : public Workable {
// Только то, что нужно!
   void work() override {} };
```

Хорошо:

```
template<typename T>
concept Worker = requires(T t) {
   { t.work() } -> std::same_as<void>;
};
template<typename T>
concept Eater = requires(T t) {
   { t.eat() } -> std::same_as<void>;
};
template<Worker W>
void doWork(W& w) {
// Работает и с людьми, и с роботами
   w.work();}
template<Eater E>
void doLunch(E& e) {
   e.eat();   // Только для тех, кто ест
}
struct Human {
   void work() {}
   void eat() {}
};
struct Robot {
// Нет метода eat()
   void work() {}
};
```

I – Разделение интерфейсов

Неудачно:

```
class Worker {  // Огромный интерфейс
public:
   virtual void work() = 0;
   virtual void eat() = 0;
   virtual void sleep() = 0;
   virtual void code() = 0;
   virtual void design() = 0;
   virtual void manage() = 0;
};
class Robot : public Worker {  // Роботу не нужны eat/sleep!
   void work() override {}
   void eat() override { throw "Robots don't eat"; }  // ПРОБЛЕМА!
   void sleep() override { throw "Robots don't sleep"; }  // ПРОБЛЕМА!
   // ...
};
```

Хорошо:

```
class Workable {
public:
   virtual void work() = 0;
};
class Eatable {
public:
   virtual void eat() = 0;
};
class Sleepable {
public:
   virtual void sleep() = 0;
};
class Human : public Workable, public Eatable, public Sleepable {
   void work() override {}
   void eat() override {}
   void sleep() override {}
};
class Robot : public Workable {
// Только то, что нужно!
   void work() override {} };
```

Хорошо:

```
template<typename T>
concept Worker = requires(T t) {
   { t.work() } -> std::same_as<void>;
};
template<typename T>
concept Eater = requires(T t) {
   { t.eat() } -> std::same_as<void>;
};
template<Worker W>
void doWork(W& w) {
// Работает и с людьми, и с роботами
   w.work();}
template<Eater E>
void doLunch(E& e) {
   e.eat();   // Только для тех, кто ест
}
struct Human {
   void work() {}
   void eat() {}
};
struct Robot {
// Нет метода eat()
   void work() {}
};
```

D – Принцип инверсии зависимостей

Неудачно:

Конкретный класс (Неудачно)

```
class Bad {
    MySQLDatabase db;  // Привязаны к конкретной БД
};
```

Хорошо:

Абстрактный класс (Правильно)

```
class Good {
    Database& db;  // Можно подставить любую БД
};
```

Хорошо:

```
template<typename DB>
class TemplateGood {
    DB db;  // Можно подставить что угодно с нужным интерфейсом
};
```

Соблюдение SOLID

```
class ReportGenerator { void generate(){} };
class ReportSaver { void save(){} };
class ReportPrinter { void print(){} };
class EmailSender { void send(){} };
```

Нарушение SOLID

```
class Report {
    void generate() {/*...*/}
    void saveToFile() {/*...*/}
    void print() {/*...*/}
    void sendEmail() {/*...*/}
};
```

Обертывание

- Паттерн, где один класс **"оборачивает"** другой, добавляя новую функциональность **без изменения исходного класса**

Паттерн — это **проверенное решение** часто встречающейся проблемы в программировании

Обертывание

Исходный класс

```
class Coffee {
public:
   virtual double cost() {
return 10.0; }
   virtual string
description() { return
"Coffee"; }
};
```

Добавление функциональности:

```
class MilkWrapper : public Coffee {
   Coffee* wrapped;
public:
   MilkWrapper(Coffee* c) : wrapped(c)
{}
   double cost() override {
// Добавляем цену молока
      return wrapped->cost() + 2.0; }
   string description() override {
      return wrapped->description() + "
+ milk"; } };
class SugarWrapper : public Coffee {
Coffee* wrapped;
public:
   SugarWrapper(Coffee* c) :
wrapped(c) {}
   double cost() override {
      return wrapped->cost() + 0.5;
   }
   string description() override {
      return wrapped->description() + "
+ sugar"; } };
```

Использование:

```
Coffee* coffee = new
Coffee();
// Обертывание
coffee = new
MilkWrapper(coffee);
// Обертывание  еще раз
coffee = new
SugarWrapper(coffee);
// "Coffee + milk + sugar"
cout << coffee-
>description();
// 12.5
cout << coffee->cost();
```

Типы оберток

1. Декоратор (Decorator) - добавляет функциональность

```
// Добавляем логирование к любому объекту
class LoggerWrapper : public Service {
    Service* wrapped;
public:
    void doSomething() override {
        log("Starting...");
        wrapped->doSomething();
        log("Finished...");
    }
};
```

Типы оберток

2. Адаптер (Adapter) - меняет интерфейс

```
// Старый интерфейс адаптируется к новому
class OldPrinter {
public:
   void printText(string s) { /* ... */ }
};

class NewPrinterAdapter : public NewPrinter {
   OldPrinter* oldPrinter;
public:
   void print(string s) override {
      oldPrinter->printText(s);  // Адаптируем вызов
   }
};
```

Типы оберток

3. Фасад (Facade) - упрощение сложной системы

```
class ComputerFacade {
    CPU cpu;
    Memory memory;
    HardDrive hd;

public:
    void start() {
        cpu.init();
        memory.load();
        hd.read();
    }
};
```

Сравнение с наследованием

```
class MilkWrapper : public Coffee {
    Coffee* wrapped;  // указатель на "завернутый" кофе
public:
    MilkWrapper(Coffee* c) : wrapped(c) {}
    double cost() override {
        return wrapped->cost() + 2.0;  // добавляем цену
молока
    }
};

class SugarWrapper : public Coffee {
    Coffee* wrapped;
public:
    SugarWrapper(Coffee* c) : wrapped(c) {}
    double cost() override {
        return wrapped->cost() + 0.5;  // добавляем цену
сахара
    }
};
```

```
// Наследование
class CoffeeWithMilk : public Coffee
{ };

// создание базового кофе
 Coffee* coffee = new Coffee();
// Добавление молока
coffee = new MilkWrapper(coffee);
// Добавление сахара
coffee = new SugarWrapper(coffee);
```

```cpp
#include <iostream>
#include <memory>
#include <string>
#include <stdexcept>
class Beverage {
public:
   virtual ~Beverage() = default;
   // Чисто виртуальные методы
   virtual double cost() const = 0;
   virtual std::string description() const = 0;
protected:
   Beverage() = default;
};
class Espresso : public Beverage {
public:
   double cost() const override {
      return 1.99;
   }
   std::string description() const override {
      return "Espresso";
   }
};

class HouseBlend : public Beverage {
public:
   double cost() const override {
      return 0.89;
   }
   std::string description() const override {
      return "House Blend Coffee";
   }
};
class Decaf : public Beverage {
public:
   double cost() const override {
      return 1.05;
   }
   std::string description() const override {
      return "Decaf Coffee";
   }
};
class CondimentDecorator : public Beverage {
protected:
   std::unique_ptr<Beverage> beverage;
public:
   CondimentDecorator(std::unique_ptr<Beverage> b)
      : beverage(std::move(b)) {
      if (!beverage) {
         throw std::invalid_argument("Beverage cannot be
null");
      }
   }
   std::string description() const override {
      return beverage->description();
   }
};
class Milk : public CondimentDecorator {
public:
   using CondimentDecorator::CondimentDecorator;
   double cost() const override {
      return beverage->cost() + 0.30;
   }
   std::string description() const override {
      return beverage->description() + ", Milk";
   }
};
class Mocha : public CondimentDecorator {
public:
   using CondimentDecorator::CondimentDecorator;
   double cost() const override {
      return beverage->cost() + 0.20;
   }
   std::string description() const override {
      return beverage->description() + ", Mocha";
   }
};
class Whip : public CondimentDecorator {
public:
   using CondimentDecorator::CondimentDecorator;
   double cost() const override {
      return beverage->cost() + 0.10;
   }
   std::string description() const override {
      return beverage->description() + ", Whip";
   }
};
class Soy : public CondimentDecorator {
public:
   using CondimentDecorator::CondimentDecorator;
   double cost() const override {
      return beverage->cost() + 0.15;
   }
   std::string description() const override {
      return beverage->description() + ", Soy";
   }
};
```

Использование

```
int main() {
  try {
    std::cout << "=== Coffee Shop Order System ===\n"
              << std::endl;
    // Заказ 1: Espresso без добавок
    auto beverage1 = std::make_unique<Espresso>();
    std::cout << "Order 1: " << beverage1->description()
          << " $" << beverage1->cost() << std::endl;
    // Заказ 2: House Blend с двойным Mocha и Whip
    auto beverage2 = std::make_unique<Whip>(
      std::make_unique<Mocha>(
        std::make_unique<Mocha>(
          std::make_unique<HouseBlend>()
        )
      )
    );
    std::cout << "Order 2: " << beverage2->description()
          << " $" << beverage2->cost() << std::endl;
    // Заказ 3: Decaf с Soy, Mocha, Whip
    auto beverage3 = std::make_unique<Whip>(
      std::make_unique<Mocha>(
        std::make_unique<Soy>(
          std::make_unique<Decaf>()
        )
      )
    );
std::cout << "Order 3: " << beverage3->description()
          << " $" << beverage3->cost() << std::endl;
    // Заказ 4: House Blend со всеми добавками
    auto beverage4 = std::make_unique<Whip>(
      std::make_unique<Mocha>(
        std::make_unique<Soy>(
          std::make_unique<Milk>(
            std::make_unique<HouseBlend>()
          )
        )
      )
    );
    std::cout << "Order 4: " << beverage4->description()
          << " $" << beverage4->cost() << std::endl;
    // Демонстрация разных комбинаций
    std::cout << "\n=== Price Breakdown ===\n"
              << std::endl;
    // Простая демонстрация цепочки
    auto simple = std::make_unique<HouseBlend>();
    std::cout << "Base: " << simple->description()
          << " $" << simple->cost() << std::endl;
    auto withMilk = std::make_unique<Milk>(std::move(simple));
    std::cout << "With Milk: " << withMilk->description()
          << " $" << withMilk->cost() << std::endl;
    auto withMocha =
std::make_unique<Mocha>(std::move(withMilk));
    std::cout << "With Mocha: " << withMocha->description()
          << " $" << withMocha->cost() << std::endl;

  } catch (const std::exception& e) {
    std::cerr << "Error: " << e.what() << std::endl;
    return 1;
  }
  return 0; }
```

```
=== Coffee Shop Order System ===
Order 1: Espresso $1.99
Order 2: House Blend Coffee, Mocha, Mocha, Whip $1.39
Order 3: Decaf Coffee, Soy, Mocha, Whip $1.5
Order 4: House Blend Coffee, Milk, Soy, Mocha, Whip $1.64
=== Price Breakdown ===
Base: House Blend Coffee $0.89
With Milk: House Blend Coffee, Milk $1.19
With Mocha: House Blend Coffee, Milk, Mocha $1.39
```

Преимущества

- **Не меняется исходный код** (работает с чужими/ legacy* классами)
- **Добавляется функциональность динамически** (во время выполнения)
- **Соблюдается Open/Closed Principle** (классы закрыты для изменений, открыты для расширения)
- **Замена наследования** (композиция вместо наследования)

*старый код , уже используемый в проекте